Technical Build Manual & Troubleshooting Log: Portfolio & AI Integration

- **Author:** Jeremy Dowdy
- **Document Version:** 1.0 (Portfolio Site Live / AI Chatbot Component Active Development)

1. System Architecture Overview
The system is architected as a decentralized, serverless web application. To prevent exposure of private credentials on the client-side, the infrastructure relies on a three-tier model:

1. **Presentation Layer:** A static single-page portfolio (`index.html`) hosted via GitHub Pages, utilizing modern CSS styles and embedded client-side JavaScript.
2. **Edge Proxy Layer: Cloudflare Workers**, acting as an intermediate serverless bridge to process cross-origin requests and manage secure environment routing.
3. **Intelligence Layer: Google Gemini API (gemini-1.5-flash)**, providing dynamic natural language processing and context-aware response generation.

2. Iterative Build Log: The Frontend & Structural Evolution
Iteration 1: Establishing the Foundation

- **Objective:** Create a high-performance profile highlighting an IT professional and U.S. Marine Corps Veteran background.
- **Implementation:** Built a single file structure incorporating a responsive viewport, CSS backdrop filters (`backdrop-filter: blur`), and an absolute background layer (`background-image`).

Iteration 2: Asset Management & Logo Rendering Hiccups

- **The Problem:** Attempting to render official tech-stack logos via external public image URLs (such as Wikipedia SVGs) triggered hotlinking blocks and cross-origin resource restrictions. This caused browsers to display broken asset placeholders ("grey boxes with question marks").
- **The Resolution:** Bypassed external asset dependencies by pivoting to a clean, highly reliable text-styled footer (`Google Cloud • Gemini AI • Cloudflare`), ensuring visual integrity across all modern browsers.

3. The AI Chatbot Build Manual: Hiccups, Architecture, and Roadblocks
The primary engineering hurdle of this build was shifting away from traditional, rigid rule-based response bots (hardcoded conditional string matching) toward an autonomous, context-aware LLM interface.
Step 3.1: Establishing the Cloudflare Worker Bridge
To connect the frontend chat window safely to Google Gemini without leaking API secrets, a Cloudflare Worker was written to proxy incoming `POST` requests.

- **Initial Hiccup (The Direct-Hit Crash):** Visiting the Cloudflare Worker URL directly in a browser window triggered a `GET` request. Because the worker code was expecting a structured JSON payload via `POST`, it threw a `SyntaxError: Unexpected end of JSON input`.
- **The Fix:** Implemented method verification inside the Worker script to intercept non-POST requests gracefully:

```
if (request.method !== "POST") {
  return new Response("Bridge is active.", { status: 200 });
}
```

Step 3.2: CORS (Cross-Origin Resource Sharing) Handshakes

- **The Hiccup:** Browser security policies blocked frontend requests originating from the GitHub Pages domain to the `.workers.dev` endpoint, resulting in browser console "Load Failed" exceptions.
- **The Fix:** Configured explicit `OPTIONS` preflight handling and response headers within the Cloudflare Worker:

```
"Access-Control-Allow-Origin": "*"
"Access-Control-Allow-Methods": "POST, OPTIONS"
```

Step 3.3: Secret Management & The "Undefined Object" Crisis

- **The Hiccup:** Even after establishing the bridge, testing the chat returned `Error: undefined is not an object (evaluating 'json.candidates[0]')`. This indicated that while the network connection succeeded, Google's API was rejecting the payload or authorization.
- **Root Cause Analysis:** The `GEMINI_API_KEY` environment variable inside Cloudflare had initially been set as standard **"Text"** instead of an encrypted **"Secret."** Furthermore, environment variable updates in Cloudflare Workers require an explicit manual **Deploy** action to propagate to the edge.
- **The Resolution:**
  1. Deleted and re-added the API credential explicitly as a Cloudflare **Secret** named `GEMINI_API_KEY`.
  2. Triggered a manual worker redeployment.
  3. Added extensive JSON response inspection logging (`console.log`) to track payload structures in real-time.

4. Current Status & Next Steps

- **The Portfolio Website:** Fully operational, live, and optimized.
- **The AI Chatbot Component:** Active troubleshooting phase. While the edge proxy, CORS compliance, and secret-key handshakes are resolved, fine-tuning autonomous prompt safety rules, response payload stability, and conversational guardrails is ongoing.

(Note: Because the AI chatbot implementation involves its own distinct set of edge-security and prompt-engineering complexities, a separate technical documentation file will be dedicated exclusively to the chatbot's final operational state once stabilization is complete.)
For a visual demonstration of similar edge-AI integration patterns, you can review this overview Building an AI Calculator with Webflow: Integrating Gemini API, GitHub, and Cloudflare Workers.